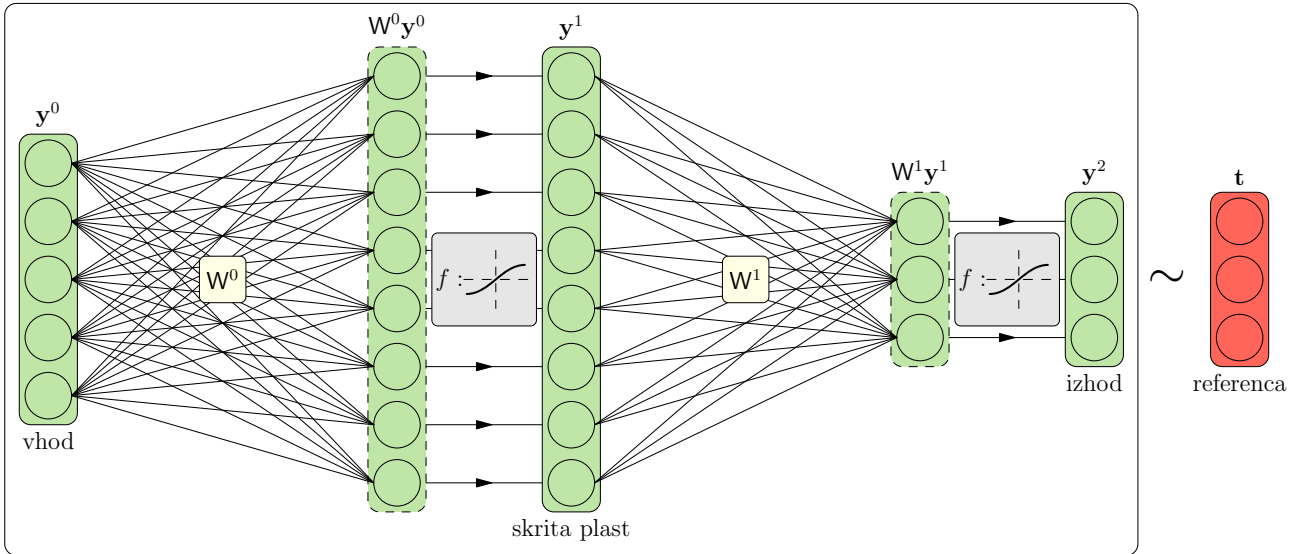


13. naloga – Nevronske mreže

Modeli nevronske mreže so verige preslikav med vektorji $\mathbf{y}^\ell$ (ℓ -ta “plast” nevronov) v obliki $\mathbf{y}^{\ell+1} = f(W^\ell \mathbf{y}^\ell)$, kjer so W^ℓ linearne preslikave (v splošnem nekvadratne matrike)¹, f pa je nelinearna aktivacijska funkcija, ki deluje na vektor po komponentah. Nevronske mreže so tip univerzalnih funkcij, ki v principu lahko predstavljajo poljubno funkcijsko zvezo med vhodnim vektorjem $\mathbf{y}^0$ (običajno veliko komponent) in izhodnim vektorjem $\mathbf{y}^L$ (običajno malo komponent), kjer je L število preslikav med plastmi. Aktivacijska funkcija je običajno sigmoidna funkcija, ki poreže signal na interval $(-1, 1)$, v našem primeru bomo vzeli $f(x) = \tanh(x)$. Glede na namen ločimo različne vrste nevronske mreže, ki se razlikujejo po številu skritih plasti (in s tem številu matrik z utežmi), številu nevronov v skritih plasteh (dimenzije vektorjev), ter načina povezav (ali so matrike polne, ali morda bločne glede na izbrano lokalno povezanost, kot v primeru konvolucijskih nevronske mreže). Nevronske mreže se pogosto uporabljajo za klasifikacijo, kjer izhodne komponente interpretiramo kot odgovore da/ne o pripadnosti izbranim kategorijam. Vsa zapletena heuristika odločanja se skriva v koeficientih utežnih matrik. “Deep learning” mreže se sklicujejo na modele z večjim številom skritih plasti, ki hierarhično združujejo prepoznane detajle v vedno kompleksnejše pojme.

Izvedenost rezultata nevronske mreže je trivialna operacija, zahtevni del je inverzni problem – določitev parametrov W^ℓ nevronske mreže – saj poznamo le željeni rezultat za nekaj primerov vhoda. Ta problem rešujemo s treniranjem nevronske mreže na veliki količini vhodnih vektorjev $\mathbf{y}^0$ z znanim izidom $\mathbf{t}$. Primer take mreže z eno skrito plastjo je prikazan spodaj:



Odstopanje med znanim izidom in rezultatom nevronske mreže, definirano s funkcijo $E = \frac{1}{2} \|\mathbf{y}^L - \mathbf{t}\|^2$ (“loss function”), minimiziramo z vzvratno propagacijo napake, ki je poseben primer minimizacije z gradientnim spustom. Definirajmo residual, ki se rekurzivno propagira z zadnje plasti nevronov proti prvi,

$$\delta^{L-1} = -(\mathbf{y}^L - \mathbf{t}) \odot f'(W^{L-1} \mathbf{y}^{L-1}) \quad (1)$$

$$\delta^\ell = \delta^{\ell+1} W^{\ell+1} \odot f'(W^\ell \mathbf{y}^\ell), \quad (2)$$

kjer $\odot$ ponazarja množenje istoležnih komponent. Ker velja $\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$, lahko zmanjšamo število računskih operacij, saj smo vse $\tanh(W^\ell \mathbf{y}^\ell)$ že izračunali med izvedenostjo mreže od leve proti desni.

¹Vsem plastem razen zadnji lahko dodamo komponento, ki ne pride iz vhodnih podatkov ali prejšnje plasti, ampak je vedno enaka 1. S tem vsem linearnim preslikavam dodamo še nehomogeni člen (“bias”).

Popravek matrikam uteži, ki ga izvedemo na vsakem učnem primeru, potem zapišemo kot

$$W_{ij}^\ell \mapsto W_{ij}^\ell + \eta \delta_i^\ell y_j^\ell,$$

pri čemer je η hitrost učenja (korak gradientnega spusta). Primerna vrednost je na primer $\eta = 0.01$. Začetne uteži W^ℓ so običajno matrike Gaussovih naključnih števil z disperzijo $\sigma_\ell^2 = 2/\dim(\mathbf{y}^\ell)$.

1. V datotekah `train-images-idx3-ubyte` in `train-labels-idx1-ubyte` najdeš nabor črnobelih slik ročno napisanih števk od 0 do 9 s pripadajočimi oznakami 0 – 9. Skonstruiraj in nauči nevronska mrežo s tremi plastmi – na prvo plast pripeljemo vhodne podatke ($28^2 = 784$ pikslov slike, reskaliranih na interval med 0 in 1), skrita plast je lahko kar enake dimenzije kot prva, izhodna plast pa predstavlja 10 izhodov, za vsako števko enega, katerih ciljne vrednosti so 1 za pravo števko in -1 za napačne. Poskusiš lahko tudi z različnimi dimenzijami skritih plasti ali za več skritih plasti, ter z različnimi hitrostmi učenja.

Tekom učenja spremljaj napako E ter delež pravilno določenih števk. Nekaj podatkov (npr. desetino) prihrani za testiranje po koncu učenja. Katere številke algoritem največkrat zamenjuje med sabo in katere najslabše določi? Bi sami naredili isto napako?

Nevronska mreža se lahko odziva nepredvidljivo, če ji ponudimo podatke, ki močno odstopajo od podatkov, na katerih je bila mreža trenirana. Kakšen izid dobimo, če na vhod pošljemo slike, ki ne predstavljajo števk?

2. Pri učenju mreže smo minimizirali napako z iterativnim popravljanjem uteži W^ℓ . Na že naučeni mreži pa lahko vprašamo, kateri *vhod* minimizira napako na izhodu za *izbrani izhod*. Z enakim postopkom posrednega odvajanja, s katerim smo določili popravke uteži, dobimo iteracijski popravek za vhodni vektor

$$\mathbf{y}^0 \mapsto \mathbf{y}^0 + \eta \delta^0 W^0.$$

pri čemer do δ^0 pridemo s pomočjo rekurzije (2). To iteracijsko shemo ponavljamo, dokler napaka na izhodu ne doseže zadovoljivo majhne vrednosti. Začetni približek za sliko $\mathbf{y}^0$ je v našem primeru lahko kar črna slika, na katero želimo, da nevronska mreža nariše izbrano števko glede na naučene značilnosti. Ta postopek je znan tudi pod frazo “deep dream” in ga lahko uporabimo kot vpogled v lastnosti, ki jih mreža zazna kot pomembne za klasifikacijo.

Nabor slik je shranjen binarno v eni sami datoteki, s krajšo glavo, ki ji sledijo bajti, ki predstavljajo sivine 28×28 slike. Podobno velja za oznake (števka med 0 in 9) za istoležne slike. Podrobnosti formata so navedene na povezavi <http://yann.lecun.com/exdb/mnist/>. Kot pomagalo najdete na povezavi `neuralnet-source.tar.gz` vzorčne implementacije branja teh datotek v jezikih Python, c in c++.